StyleVision AI: Two-Stage Apparel Detection and Classification Pipeline

PROJECT REPORT: StyleVision AI

Student Name: Akshat Jha | 3rd Year, B.Tech

Institution: VIT Bhopal University

Repository: https://github.com/AkshatJ24/StyleVision-AI

1. The Problem and Why it matters

For the domain of Computer Vision, the standard image classification models are flawed by a fundamental limitation known as the "Naive Classifier" problem. A standard Convolutional Neural Network (CNN) model that is only given clothing data and is expected to classify clothing images will, upon seeing an image of a coffee cup or a chessboard, have to guess what it is and will frequently report false positives.

Why it matters: In the real world, false positives are a huge problem in applications such as e-commerce image tagging, smart retail inventory management, and virtual wardrobe management. An AI system must be "defensive." It must have the spatial understanding to confirm the presence of the object before it attempts to classify the object's attributes.

2. My Approach to Solving It

In order to develop a fault-tolerant system, I shifted my focus from a singular model-based approach and implemented a Two-Stage Hybrid Pipeline, where I have directly incorporated concepts from Module 3, which deals with 'Feature Extraction and Image Segmentation,' from the Computer Vision course syllabus.

- **Stage 1: The Gatekeeper - Object Detection & Segmentation**: I employed YOLOv8 for the initial scan of the image. However, instead of detecting objects, I employed it for dynamic region of interest (ROI) cropping. This is a more advanced level of segmentation, where the clothing item is cropped, and the background noise is eliminated.

- **Stage 2: The Brain - Deep Feature Extraction**: The cropped region is then employed for a MobileNetV2-based model, where I employed Transfer Learning from a dataset of 15,000+ images of clothing items, classified into 10 categories.

By chaining these models, the pipeline acts sequentially: ***"Is there clothing here? If yes, what exactly is it?"***

3. Key Decisions Made

- **Implementing a "Hard Reject" Filter**: I configured the YOLO layer to actively seek out common non-apparel items (class IDs for cups, chairs, etc.). If detected with high confidence, the pipeline is terminated, and a "Non-Apparel Item" rejection is outputted. This ensures the classification model does not waste resources classifying invalid data.

- **Dual-Confidence Thresholding**: For the confidence thresholding, I chose to implement a dynamic scoring guardrail.

If YOLO is able to crop the image, the classification threshold is set to 70%. However, if YOLO fails to crop the image, the classification threshold is set to 85%. This way, the AI is not "unsure" and can output "Unknown Object" in all scenarios.

- **Prioritizing a Local-First Architecture**: Due to the high computational requirements for running YOLO and TensorFlow simultaneously, I prioritized the local-first architecture. For the Flask server, I optimized the environment to load the heavy weights into memory only once, during the server "cold start." This ensures that all inferences run in under 200 milliseconds.

4. Challenges Faced

- **Handling Out-of-Distribution (OOD) Data**: Early in development, the model struggled heavily with abstract images and random internet photos. I had to iteratively test edge cases (like the "Chessboard Challenge") to properly calibrate the YOLO confidence filters and ensure the system knew when to reject an image.

- **Dependency Management & Model Size**: Managing a local setup with massive datasets (over 1GB of training data) alongside GitHub's size limits was a major logistical challenge. I had to learn how to properly configure .gitignore files to keep the local dataset hidden while pushing the lightweight .txt instructions and UI code to the public repository. Additionally, balancing library versions (like switching to tensorflow-cpu for deployment viability) required careful environment management.

5. What I Learned

Building StyleVision AI closed the gap between theoretical syllabus ideas and real-world MLOps.

- **System Design Over Model Tuning**: I discovered that a system utilizing two lightweight models (YOLO Nano and MobileNet V2) and smart logic is significantly more robust and accurate than attempting to train one massive monolithic network.

- **The Importance of Defensive AI**: I gained a newfound respect for edge cases. Designing a UI to correctly display "Unknown Item" to the user, as opposed to crashing or lying, is an important part of software development.

- **End-to-End Pipeline Execution**: Expanding beyond the world of Jupyter Notebooks, I learned how to utilize deep learning models to build a cohesive Flask web application, from raw file upload to real-time memory management and frontend rendering.